

Numerische Methoden

Theorie-Lernskript – alle Themen kompakt erklärt

Grundlagen · Fehleranalyse · Nullstellen · Optimierung · DGL · Integration/Interpolation · Fourier ·
Modelltraining

Legende der Boxen: ■ Definition ■ Intuition ■ Satz/Formel ■ Stolperfalle (richtige Form direkt gezeigt)

Inhaltsverzeichnis

1 Grundlagen & Zahlendarstellung	1
1.1 Diskretisierung & Brute Force	1
1.2 Gleitkomma (floating point)	1
1.3 Festkomma (fixed point)	2
1.4 Rundung, Auslöschung & Fehlerarten	2
2 Fehleranalyse: die vier Eigenschaften	2
3 Nullstellen: Newton- & Sekantenverfahren	3
4 Optimierung	3
5 Differenzialgleichungen (Anfangswertprobleme)	4
6 Numerische Integration & Interpolation	5
6.1 Interpolation	5
6.2 Romberg & Monte-Carlo	6
7 Fourier-Analyse & FFT	6
8 Modelltraining aus ersten Prinzipien	6

1 Grundlagen & Zahlendarstellung

Warum Numerik?

Viele Probleme haben *keine* geschlossene Lösung (z. B. $\int e^{-x^2} dx$, $x = \cos x$, allgemeine Nullstellen ab Grad 5) oder sind analytisch zu aufwendig. **Numerik** liefert *näherungsweise* Zahlenergebnisse mit endlich vielen Rechenschritten – Preis: jedes Ergebnis hat einen *Fehler*, den man verstehen und kontrollieren muss.

1.1 Diskretisierung & Brute Force

Ein kontinuierliches Problem wird durch endlich viele Stützstellen ersetzt. Intervall $[a, b]$ in N Teile: **Schrittweite** $h = \frac{b-a}{N}$, Gitterpunkte $x_i = a + i h$ ($i = 0, \dots, N$, also $N+1$ Punkte). *Grid Search / Brute Force*: alle Gitterpunkte durchprobieren und den besten nach einem *Fehlermaß* wählen.

Grid Search für $Ax = c$

Fehlermaß = **Residuum** $E(\mathbf{x}) = \|A\mathbf{x} - \mathbf{c}\|$ (z. B. Summe der Beträge oder Quadrate). Über das Gitter minimieren. *Trifft die exakte Lösung nur, wenn diese zufällig auf einem Gitterpunkt liegt* – sonst bleibt ein Diskretisierungsfehler. Verbesserung: feineres Gitter oder lokale Verfeinerung um den besten Kandidaten.

1.2 Gleitkomma (floating point)

Gleitkommazahl

$x = (-1)^s m \cdot 2^e$ mit **Vorzeichen** s , **Mantisse** m (signifikante Stellen), **Exponent** e (Skalierung, gespeichert mit *Bias*: $e = E - \text{Bias}$). *Normalisiert*: $m = 1.b_1b_2\dots b_t$ mit *implizitem* führenden 1-Bit (1 Gratis-Bit).

Maschinenepsilon

$\varepsilon_{\text{mach}} = 2^{-t}$ (t = Anzahl gespeicherter Mantissen-Nachkommabits) ist der Abstand von 1 zur nächstgrößeren darstellbaren Zahl – die *kleinste relative Auflösung*. Relativer Rundungsfehler stets $\leq \frac{1}{2}\varepsilon_{\text{mach}}$.

Trade-off Mantisse $\leftrightarrow$ Exponent (bei fester Bitzahl)

Mehr Mantissenbits $\Rightarrow$ kleineres $\varepsilon \Rightarrow$ feinere *Auflösung* (resolution). **Mehr Exponentenbits** $\Rightarrow$ größerer *Wertebereich* (dynamic range, größte/kleinste Beträge). Man tauscht also stets *Präzision* gegen *Reichweite*.

1.3 Festkomma (fixed point)

f Nachkommabits, **Auflösung** $= 2^{-f}$ (konstant). Speicherwert $= \text{Zahl} \cdot 2^f$ (ganzzahlig), größte vorzeichenlose Zahl mit k Gesamtbits $= \frac{2^k - 1}{2^f}$.

Festkomma vs. Gleitkomma – konstanter *absoluter* vs. *relativer* Abstand

Festkomma: Nachbarzahlen haben *überall denselben* Abstand 2^{-f} (gleichmäßiges Raster). Gleitkomma: *relativer* Abstand konstant ($\approx \varepsilon$), d. h. der *absolute* Abstand wächst mit der Größenordnung von x (eng bei kleinen, weit bei großen Zahlen).

1.4 Rundung, Auslöschung & Fehlerarten

Auslöschung (catastrophic cancellation)

Subtrahiert man zwei *fast gleiche* Zahlen, löschen sich die führenden Stellen weg; übrig bleiben wenige, schon verrauschte Stellen $\Rightarrow$ *relativer* Fehler explodiert. Beispiel: $(1 + \delta) - 1$ verliert δ komplett, falls $\delta < \frac{1}{2}\varepsilon$ (dann Ergebnis 0, Division dadurch $\rightarrow \infty$).

Gegenmittel: umformen statt subtrahieren

$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$ – rechts wird *addiert*, keine Auslöschung mehr. Allgemein: kritische Differenzen algebraisch in Summen/Produkte umschreiben.

Die drei Fehlerarten

- **Modellfehler:** das Modell vereinfacht die Realität (z. B. Reibung/Luftwiderstand vernachlässigt).
- **Abschneide-/Diskretisierungsfehler:** ein Grenzprozess wird abgebrochen (Reihe nach endlich vielen Gliedern; Ableitung $\rightarrow$ Differenzenquotient). Skaliert mit h .
- **Rundungsfehler:** endliche Stellenzahl der Maschine (z. B. $\frac{10}{3} \rightarrow 3,333$).

Absoluter Fehler $|x - \tilde{x}|$; **relativer** $\frac{|x - \tilde{x}|}{|x|}$ (skaleninvariant, „gültige Stellen“; bei $x \approx 0$ nur absolut sinnvoll).

2 Fehleranalyse: die vier Eigenschaften

Hut & Tilde

f = wahres Problem, $\hat{f}$ = der *Algorithmus* (gestörte Rechenvorschrift), $\tilde{x}$ = *gestörte Eingabe*. Die vier Eigenschaften vergleichen diese vier Größen paarweise.

Kondition, Stabilität, Konsistenz, Konvergenz

- **Kondition** $\kappa = |f(x) - f(\tilde{x})|$: wie stark reagiert das *Problem* auf Eingabestörung. Klein = gut, groß = schlecht konditioniert. *Eigenschaft des Problems*.
- **Stabilität** $|\hat{f}(\tilde{x}) - \hat{f}(x)| \leq s |\tilde{x} - x|$: wie stark verstärkt der *Algorithmus* Störungen. *Eigenschaft des Verfahrens*.
- **Konsistenz** $|\hat{f}(x) - f(x)| \leq c$: wie gut nähert der Algorithmus bei *exakter* Eingabe.
- **Konvergenz** $|\hat{f}(\tilde{x}) - f(x)| \rightarrow 0$: Gesamtfehler aus gestörter Eingabe.

Kernzusammenhang

Konsistenz + Stabilität $\Rightarrow$ Konvergenz. Quantifizierung über $|f'(x)|$: $\kappa \approx |f'(x)| \delta x$ – dort, wo f *flach* ist (kleine Steigung), gut konditioniert; wo f *steil* ist, schlecht.

Zwei Verwechslungen, die Punkte kosten

- 1) **Kondition $\neq$ Stabilität:** Kondition gehört zum *Problem*, Stabilität zum *Algorithmus*. Ein stabiler Algorithmus kann ein *schlecht konditioniertes* Problem **nicht** retten – er erzeugt nur keine *zusätzlichen* Fehler.
- 2) **Kleineres h ist *nicht* automatisch besser:** feineres h senkt den Konsistenz-/Diskretisierungsfehler, aber unterhalb eines optimalen h *dominiert der Rundungsfehler* (und Eingangsrauschen) – der Gesamtfehler steigt wieder. Es gibt ein optimales h .

3 Nullstellen: Newton- & Sekantenverfahren

Newton-Verfahren – Herleitung aus Taylor

Taylor 1. Ordnung: $f(x) \approx f(x_n) + f'(x_n)(x - x_n)$. Diese *Tangente* = 0 setzen und nach x lösen:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Newton iteriert also die Nullstelle der Tangente.

Konvergenz von Newton

Quadratische Konvergenz $|e_{n+1}| \leq C|e_n|^2$ nahe einer *einfachen* Wurzel x^* : die Zahl korrekter Stellen *verdoppelt* sich pro Schritt ($10^{-1} \rightarrow 10^{-2} \rightarrow 10^{-4} \rightarrow 10^{-8}$). **Konvergenzkriterium:** x_0 nah genug an einfacher Wurzel, $f \in C^2$, $f'(x^*) \neq 0$.

Newton-Versagensfälle

- $f'(x_n) = 0$: waagrechte Tangente $\Rightarrow$ *Division durch 0* (Verfahren bricht ab).
- Schlechter Startwert $\Rightarrow$ Divergenz oder Sprung ins falsche Becken.
- *Zyklus*: Iteration pendelt periodisch und konvergiert nie.

Finite Differenz & Sekantenverfahren

Ist f' unbekannt/teuer: **Vorwärtsdifferenz** $f'(x_0) \approx \frac{f(x_0+h)-f(x_0)}{h}$, Fehler $\mathcal{O}(h)$ (Taylor-Restterm $\frac{h}{2}f''$). Einsetzen $\rightarrow$ **Sekantenverfahren**

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Ableitungsfrei (Ordnung $\approx 1,618$). Vorteil: kein f' nötig. Nachteil: langsamer als Newton, braucht zwei Startwerte, Nenner kann ≈ 0 werden.

4 Optimierung

Lokal vs. global, Konvexität

Lokales Minimum: kleinster Wert in einer *Umgebung*; **globales**: kleinster Wert auf der *ganzen* Domäne. Lokale Verfahren folgen nur lokaler Information und laufen ins *nächste* Tal – bei *multi-modalen* Landschaften (viele Minima, z. B. Shekel-Foxholes) verfehlen sie das globale.

Konvexität als Glücksfall

Ist f *konvex* ($f'' \geq 0$ überall), gibt es nur *ein* Minimum – dann ist jedes lokale zugleich global, lokale Verfahren genügen. Schwierig wird es erst im *nicht-konvexen* Fall.

Newton fürs Optimum & Gradientenabstieg

Optimum: $f'(x) = 0$. Newton auf f' :

$$x_{n+1} = x_n - \frac{f'(x_n)}{|f''(x_n)|}.$$

Der *Betrag* $|f''|$ erzwingt einen Schritt *immer bergab* (auch in konkaven Regionen $f'' < 0$, wo reines Newton bergauf zu einem Maximum liefe). Klassifikation via 2. Ableitung: $f'' > 0 \Rightarrow$ Minimum, $f'' < 0 \Rightarrow$ Maximum.

Gradientenabstieg: $x_{n+1} = x_n - \eta f'(x_n)$. Lernrate η zu *groß* $\Rightarrow$ Überschießen/Oszillation/Divergenz; zu *klein* $\Rightarrow$ sehr langsam.

Globale Strategien

- **Random Restarts:** viele zufällige Starts; Erfolgswahrscheinlichkeit nach K Starts $P = 1 - (1-p)^K$ (p = Trefferchance pro Start).
- **Simulated Annealing:** akzeptiert Verschlechterung $\Delta E > 0$ mit $e^{-\Delta E/T}$. Hohe Temperatur T = viel Exploration (verlässt lokale Minima), Abkühlen $T \downarrow$ = Exploitation.
- **Basin Hopping:** zufälliger Sprung + lokale Minimierung, springt zwischen „Becken“.

Raue Landschaft & Glättung – Wahl von h

Ein hochfrequenter Störterm wie $\frac{1}{2} \sin(4\pi x)$ erzeugt viele Mikro-Minima; lokale Optimierer bleiben im ersten stecken. **Glätten** (Fenster-Mittel/Integration über eine Periode) bügelt die Oszillation aus. Wähle $h \approx \text{Periode}$ des Störterms: $\sin(4\pi x)$ hat Periode $\frac{2\pi}{4\pi} = \frac{1}{2} \Rightarrow h \approx \frac{1}{2}$.

5 Differenzialgleichungen (Anfangswertprobleme)

Euler-Verfahren

AWP $y' = f(t, y)$, $y(t_0) = y_0$. **Explizit:** $y_{n+1} = y_n + h f(t_n, y_n)$ (direkt, billig). **Implizit:** $y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$ (Gleichung lösen, teurer, viel stabiler).

Runge-Kutta & Butcher-Tableau

RK kombiniert mehrere Steigungs-Auswertungen. **RK4:**

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1), \quad k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2), \quad k_4 = f(t_n + h, y_n + h k_3).$$

Butcher-Tableau $\begin{array}{c|c} c & A \\ \hline & b \end{array}$: Knoten c_i (Zeitpunkte), Gewichte b_i (Endmischung), Matrix a_{ij} (Stufenkopplung). *Explizit* $\Leftrightarrow A$ streng untere Dreiecksmatrix. Ordnungsbedingungen u. a. $\sum_i b_i = 1$, $\sum_i b_i c_i = \frac{1}{2}$.

Ordnung, Stabilität, Steifheit

Konvergenzordnung p : globaler Fehler $\mathcal{O}(h^p)$ (Euler $p=1$, RK4 $p=4$). RK4 ist bei gleichem h *drastisch* genauer (4 Auswertungen, dafür viel größere Schritte möglich). **Stabilität** (explizit Euler, $y' = -\lambda y$): stabil nur für $0 < h < \frac{2}{\lambda}$. **Steifheit**: stark unterschiedliche Zeitskalen zwingen explizite Verfahren zu winzigen $h \Rightarrow$ implizite Verfahren nötig.

6 Numerische Integration & Interpolation

Newton-Cotes-Idee

Funktion durch ein Interpolationspolynom über äquidistante Stützstellen ersetzen und *dieses* exakt integrieren. **Exaktheitsgrad** = höchster Polynomgrad, den die Regel noch exakt integriert.

Die Quadraturregeln (auf $[a, b]$, $h = b - a$, $m = \frac{a+b}{2}$)

Regel	Formel	Exaktheit
Rechteck	$h f(a)$ bzw. $h f(b)$	0
Mittelpunkt	$h f(m)$	1
Trapez	$\frac{h}{2}[f(a) + f(b)]$	1
Simpson	$\frac{h}{6}[f(a) + 4f(m) + f(b)]$	3
3/8	$\frac{3h}{8}[f_0 + 3f_1 + 3f_2 + f_3]$	3
Boole	$\frac{2h}{45}[7f_0 + 32f_1 + 12f_2 + 32f_3 + 7f_4]$	5

Der **Mittelpunkt** ist das beste Rechteck: er mittelt die Krümmung (Über-/Unterschätzung heben sich auf) $\rightarrow \mathcal{O}(h^2)$ statt $\mathcal{O}(h)$.

Zusammengesetzte Gewichte – nur die *inneren* Punkte doppeln

Für $[0, 2]$ mit Punkten f_0, f_1, f_2 ($h = 1$):

$$\text{Trapez: } \frac{h}{2} [f_0 + 2f_1 + f_2], \quad \text{Simpson: } \frac{h}{3} [f_0 + 4f_1 + f_2].$$

Genau ein mittlerer Term, mit Gewicht 2 (Trapez) bzw. 4 (Simpson) – *nicht* $f_1 + f_2$ zusammenfassen und *nicht* zusätzliche Terme erfinden. Merksatz Simpson: „1, 4, 1 geteilt durch 3“.

Warum Simpson Grad 3 exakt schafft

Simpson legt nur eine *Parabel* (Grad 2) an, ist aber durch *Symmetrie* exakt bis Grad **3**: der kubische Fehleranteil hebt sich über das symmetrische Intervall auf.

6.1 Interpolation

Lagrange-Polynom

Durch $n+1$ Punkte (x_i, y_i) mit verschiedenen x_i gibt es *genau ein* Polynom vom Grad $\leq n$:

$$p(x) = \sum_i y_i L_i(x), \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Jedes L_i ist 1 in x_i und 0 in allen anderen Knoten. (Alternativ: Newton-Schema der dividierten Differenzen – selbes Polynom, leichter erweiterbar.)

Runge-Phänomen $\Rightarrow$ „stop at Simpson“

Interpolation *hohen Grades* auf gleichmäßigen Knoten oszilliert stark an den Rändern (Fehler wächst!). **Konsequenz:** *nicht* den Polynomgrad erhöhen, sondern **zusammengesetzte Regeln niedrigen Grades** verwenden (viele kleine Trapez-/Simpson-Stücke), bzw. **Spline**-Interpolation (stückweise niedriger Grad, glatt, stabil).

6.2 Romberg & Monte-Carlo

Romberg = Richardson auf Trapez

Trapezwerte für $h, \frac{h}{2}, \dots$ berechnen und führende Fehlerterme extrapolieren: $R = \frac{4T(h/2) - T(h)}{3}$ (eliminiert den $\mathcal{O}(h^2)$ -Term) $\rightarrow$ hohe Genauigkeit aus billigen Trapezwerten.

Monte-Carlo-Integration

$\hat{I} = |\Omega| \frac{1}{N} \sum_{i=1}^N f(X_i)$, Standardfehler $SE = \frac{|\Omega| \sigma_f}{\sqrt{N}} \propto N^{-1/2}$. *SE halbieren* $\Rightarrow N \times 4$; *zehnteln* $\Rightarrow N \times 100$. Die Mittelpunktsregel ist MC mit $N = 1$.

Fluch der Dimensionen

Ein Gitter mit n Punkten pro Achse braucht in d Dimensionen n^d Punkte (exponentiell). Der MC-Fehler $\propto 1/\sqrt{N}$ ist *dimensionsunabhängig* $\Rightarrow$ in hohen Dimensionen schlägt Monte-Carlo das Gitter.

7 Fourier-Analyse & FFT

Fourier-Transformation

$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt$ zerlegt ein Zeitsignal in seine **Frequenzanteile** ($|F(\omega)|$ = Stärke der Frequenz ω). Rücktransformation $f(t) = \frac{1}{2\pi} \int F(\omega) e^{i\omega t} d\omega$ setzt das Signal wieder zusammen (beide invers zueinander).

Euler-Formel & Interferenz

$e^{i\omega t} = \cos(\omega t) + i \sin(\omega t)$, also $\cos(\omega t) = \frac{1}{2}(e^{i\omega t} + e^{-i\omega t})$. Daher liefert ein reeller Kosinus *zwei* Spektrallinien bei $\pm\omega$. **Interferenz**: ein Signal ist die *Summe* mehrerer Schwingungen; $f(t) = A_1 \cos(\omega_1 t) + A_2 \cos(\omega_2 t)$ zeigt im Spektrum **4 Peaks** (bei $\pm\omega_1, \pm\omega_2$, Höhe $\frac{A_1}{2}, \frac{A_2}{2}$).

DFT & FFT

DFT (endlich abgetastet): $X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$ – N diskrete Frequenz-Bins. Kosten direkt $\mathcal{O}(N^2)$. **FFT** spaltet rekursiv in gerade/ungerade Indizes $\Rightarrow \mathcal{O}(N \log N)$ (bei $N=10^6$ der Unterschied zwischen Sekunden und Tagen).

Spektrum lesen: Bin-Frequenz & Nyquist

Bin k entspricht Frequenz $f_k = k \cdot \frac{f_s}{N}$ (f_s = Abtastrate). **Nyquist** = $\frac{f_s}{2}$ ist die höchste eindeutig darstellbare Frequenz. Frequenzen *darüber* erscheinen fälschlich als niedrigere (**Aliasing**, Abtasttheorem verletzt). Peaks im Betragsspektrum $\Leftrightarrow$ periodische Komponenten.

8 Modelltraining aus ersten Prinzipien

Lineare Regression als Optimierungsproblem

Modell $\hat{y}(x) = wx + b$. Trainiert wird durch Minimieren des Verlusts per Gradientenabstieg – also reine Numerik-Optimierung (Abschnitt 4) auf einer Verlustfunktion.

Mittlerer quadratischer Fehler (MSE) – mit Quadrat!

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (\underbrace{wx_i + b}_{\hat{y}_i} - y_i)^2.$$

Die Abweichungen werden **quadriert** (sonst heben sich $+/-$ auf). Das Quadrat bestraft große Fehler stärker und macht L glatt/konvex in (w, b) .

Erster Loss = Varianz der Zielwerte (Quadrat nicht vergessen!)

Init $w = 0$, $b = \bar{y}$ sagt überall den Mittelwert voraus, also

$$L = \frac{1}{n} \sum_i (\bar{y} - y_i)^2 = \text{Var}(y).$$

Varianz = Mittel der *quadrierten Abweichungen*, *nicht* der Abweichungen selbst (die ergeben immer 0). Beispiel $y = (2, 4, 6)$, $\bar{y} = 4$:

$$\text{Var} = \frac{1}{3} [(-2)^2 + 0^2 + 2^2] = \frac{1}{3} (4 + 0 + 4) = \frac{8}{3} \approx 2,67.$$

Ein gedruckter Init-Loss ≈ 100 statt $\frac{8}{3}$ deutet auf einen Bug/falsche Initialisierung (oder Summe statt Mittel); ≈ 0 wäre ebenfalls verdächtig.

Numerik-Brille auf ML-Praktiken

- **Overfit-Sanity-Check:** ein überparametrisiertes Modell *kann* eine kleine Teilmenge trivial auf Loss 0bringen. Gelingt das *nicht*, liegt der Fehler *nicht* am Optimierer, sondern in Modell/Loss/Datenpipeline – analog: einen Solver an einem Problem mit *bekannter exakter Lösung* testen.
- **Mini-Batch-SGD** $\leftrightarrow$ *Monte-Carlo*: der Gradient über eine zufällige Teilmenge der Größe B ist eine MC-Schätzung; Schätzfehler $\propto 1/\sqrt{B}$.
- **Mehrere Seeds, bester Lauf** $\leftrightarrow$ *Random Restarts* (nicht-konvexe Landschaft).
- **Quantisierung (niedrige Bit-Präzision)** $\leftrightarrow$ *Präzisionswahl* im Gleitkomma: kleinstes Format nehmen, dessen Bereich & Auflösung genügen (Inferenz verträgt weniger Präzision als Training).

Ende des Lernskripts – 8 Themenblöcke, von Zahlendarstellung bis Modelltraining. Die roten **Stolperfallen**-Boxen zeigen jeweils direkt die korrekte Form.